22

BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

<<include>> ve <<extend>> Hakkındaki Gerçek: Neden Neredeyse Hiç İhtiyacınız Yok?

The Truth About <<include>> and <<extend>>: Why You Almost Never Need Them

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              <<INCLUDE>> VE <<EXTEND>> TUZAĞI VE MODERN ÇÖZÜM │
└──────────────────────────────────────────────────────────────┘

1. ESKİ YANLIŞ KULLANIM (AŞIRI MÜHENDİSLİK):
   [ Kitap Ödünç Al ] ──<<include>>──▶ [ Kullanıcıyı Giriş Yaptır ] (YANLIŞ!)
   [ Kitap Ödünç Al ] ──<<include>>──▶ [ Veritabanına Bağlan ]     (FACİA!)
   * Kimlik doğrulama bir include DEĞİLDİR; o bir ÖNKOŞULDUR (Precondition)!

2. MODERN AIUP KURALI:
   - <<include>>: SADECE gerçekten 3'ten fazla use case tarafından paylaşılan
                  ve bağımsız değeri olan ağır bir alt akış varsa kullanılır (Nadir!).
   - <<extend>>:  Yazılımı anlamayı imkansızlaştırır; AIUP'ta NEREDEYSE HİÇ KULLANILMAZ!
   - Kural: Basit tutun! Bir akış diğerini çağıracaksa referans verin: (UC-005 Reserve Book).
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski modelciler yemek yaparken her hareket için ayrı bir döküman açmaya bayılırdı: 'Yemek Pişir' use case'i, 'Tuzluğu Tut' use case'ini `\<\<include>>` eder; 'Tuzluğu Tut' ise 'Kapağı Aç'ı `\<\<include>>` ederdi. Sonunda kimse yemek pişiremez hale gelirdi! 'Giriş Yap' bir include değildir; mutfağa girmeden önce kapının anahtarla açılmış olması bir önkoşuldur (Precondition). Akıllı bir usta gibi davranın: Birbirine bağlı gereksiz oklar çizmeyi bırakın, sadece ana işi yazın!

🇬🇧 ENGLISH (ELI5)

Bureaucratic modelers loved fracturing simple recipes: 'Cook Omelet' would `\<\<include>>` 'Grasp Salt Shaker', which would `\<\<include>>` 'Unscrew Cap'. The kitchen ground to a halt under paper overload. Authentication is not an included sub-routine; having unlocked the kitchen door is a static Precondition. In the AI era, procedural spaghetti arrows between ellipses confuse AI parsers. Omit them.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

<<include>> ve <<extend>> İlişkilerinin Çöküşü: UML tarihindeki en büyük kafa karışıklığı ve zaman israfı bu iki anahtar kelimeden doğmuştur. Cockburn 2001'de açıkça uyarmıştı: 'Alt use case'ler pahalıdır; yalnızca yeniden kullanılabilirlik veya aşırı okunabilirlik gerektiren durumlarda mazur görülebilir. Pratikte bu kural şuna indirgenir: Neredeyse hiçbir zaman!' AI Unified Process'te bu kural daha da katılaşır: 1) Kimlik Doğrulama Bir Include Değildir: 'Kullanıcı Girişi' (Log In) asla bir include olarak her senaryoya bağlanamaz; o bir Önkoşuldur (Preconditions: The member is signed in). Önkoşul olarak yazıldığında bir rota korumasına (Route Guard) dönüşür ve akışın içinde tekrar tekrar doğrulanmaz. 2) <<extend>> Yasağı: Extend ilişkisi akışın dışarıdan bir şarta bağlı olarak use case'e enjekte edilmesidir; akışın nerede dallandığını gizler ve hem insanı hem AI'ı şaşırtır. AIUP'ta extend yerine Alternatif Akışlar (A1, A2) kullanılır. 3) Handoff by Reference: Eğer bir use case başka bir use case'i tetikleyecekse, ok çizmek yerine alternatif akışın içine parantez içinde link verilir: `3. If the member accepts, system records reservation (UC-005 Reserve Book).` Hafif, temiz ve sıfır karmaşıklık!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Deconstruction of <<include>> and <<extend>>: UML literature generated decades of futile debate over relationship arrows. Cockburn's original rule of thumb: sub-use cases carry immense overhead; they are justified strictly by cross-functional reuse. In practice, this heuristic collapses to: almost never. In the AI Unified Process: 1) Authentication is never an Include: Injecting 'Log In' via `\<\<include>>` into 20 different use cases is an architectural anti-pattern. Authentication is a Precondition ('The member is signed in'). Preconditions translate cleanly into Spring Security filters or gateway route guards, bypassing in-flow redundant validations. 2) The Banishment of <<extend>>: Extension points fragment execution context across disjointed files, baffling human reviewers and confusing AI generators. All conditional deviations belong in named Alternative Flows (A1, A2). 3) Handoff by Reference: When a use case legitimately triggers another, it invokes it inline via alphanumeric citation: `(UC-005 Reserve Book)` without drawing complex UML dependency webs.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: <<include>> Yerine Önkoşul (Route Guard) ve Referans ile Çağrı Modeli
package com.maang.aiup.relationships;

public class RelationshipModernPatterns {
    // ❌ YANLIŞ: Her use case içinde kimlik doğrulama include'u çalıştırmak.
    // ✅ DOĞRU: Önkoşul (Precondition) güvenlik filtresinde çözülür:
    public record AuthenticatedUser(String id, String role) {}

    public static class CleanBorrowService {
        // Önkoşul sistem tarafından garantilenmiştir (Guarded Entry)
        public void borrow(AuthenticatedUser user, String bookId) {
            // Include yok! Doğrudan iş kuralı ve referans var:
            boolean bookAvailable = false;

            if (!bookAvailable) {
                // Handoff by reference: UC-005 Reserve Book tetiklenir!
                triggerReserveBookUseCase(user, bookId);
                return;
            }
            System.out.println("Ödünç tamamlandı.");
        }

        private void triggerReserveBookUseCase(AuthenticatedUser user, String bookId) {
            System.out.printf("UC-005 Reserve Book devreye girdi: %s için rezervasyon yapıldı.%n",
user.id());
        }
    }

    void main() {
        var service = new CleanBorrowService();
        service.borrow(new AuthenticatedUser("U1", "MEMBER"), "BK-44");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir PlantUML diyagramında 10'dan fazla <<include>> ve <<extend>> oku görüyorsanız o diyagram çöp olmuştur. Hepsini silin; kimlik doğrulamayı önkoşul yapın, dallanmaları ise alternatif akışlara taşıyın.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If a use case diagram resembles a spiderweb of \<\<include>> and \<\<extend>> arrows, the model is deeply corrupted. Purge them. Move authentication to Preconditions and push deviations into local Alternative Flows.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kimlik doğrulama bir <<include>> değildir; senaryo öncesi bir Önkoşuldur (Precondition).

• <<extend>> ilişkisi akışı gizler ve karmaşıklaştırır; AIUP'ta bunun yerine Alternatif Akışlar kullanılır.

• Başka bir use case'e geçiş, senaryo adımı içinde referans verilerek (UC-005 Reserve Book) hafifçe çözülür.

🎯 Key Architectural Takeaways

• Authentication is never an include; it is an established Precondition.

• `<<extend>>` obscures control flow; use explicit Alternative Flows instead.

• Hand off between use cases lightly by inline reference: (UC-005 Reserve Book).